Assignment 1: Analysis of SVD for Fine-Tuning LLMs

Big Data Analytics Techniques and Applications (Fall 2026) | Student ID: 315581033 (邱泓崴)
Code (public GitHub repository): https://github.com/Rewrite-StarrySky/BigData2026-Assignment1

Setup. PiSSA fine-tuning of Llama-3.2-1B on 25k MetaMathQA samples (24,975 used), all 112 target matrices (q/k/v/o/gate/up/down in 16 layers), 1 epoch, learning rate 2e-5, FP16, effective batch 128 (196 updates), max 256 new tokens; evaluation on 1,000 test questions each of GSM8K and MATH (accuracy = correct / 1,000); Kaggle 2×T4, seed 42. Every number below comes from a single run.

Question 1 (1%): Accuracy vs. target rank

(a) Report the accuracy of the original Llama-3.2-1B on GSM8K and MATH. (b) Fine-tune with target rank 16, 64 and 256 (component = default) and report the accuracy. (c) Provide observations and explanations on the performance trends with different target ranks for both datasets.

Accuracy	Original LLM	Fine-Tuned (RANK = 16)	Fine-Tuned (RANK = 64)	Fine-Tuned (RANK = 256)
GSM8K	0.035	0.157	0.204	0.274
MATH	0.001	0.038	0.046	0.054

(c) Observations and explanations

- **Fine-tuning helps substantially.** GSM8K rises from 0.035 to 0.157–0.274 (4.5×–7.8×) and MATH from 0.001 to 0.038–0.054. The original model rarely ends with a parseable final answer (956/1,000 MATH and 15/1,000 GSM8K outputs have none); MetaMathQA fine-tuning teaches both the step-by-step style and the "The answer is: …" format (unparsed after fine-tuning: GSM8K 0, MATH 456–543).
- **Accuracy increases monotonically with rank on both datasets.** GSM8K: +4.7 pts (16→64) and +7.0 pts (64→256); MATH: +0.8 and +0.8 pts. A larger rank gives more trainable parameters (11.3M / 45.1M / 180.4M = 0.9% / 3.5% / 12.7% of the model) and a larger principal subspace to adapt (Q4: ranks 16 / 64 / 256 retain only 15–35% / 36–51% / 70–76% of the energy of q_proj), so more of the fine-tuning signal can be absorbed. The final training loss falls accordingly (0.455 → 0.414 → 0.375) and the GSM8K gain has not saturated at r = 256.
- **GSM8K improves far more than MATH.** MATH requires longer multi-step derivations and more complex answer formats, which a 1B model trained on 25k examples for one epoch only partly acquires. In addition, with the 256-token limit 456–543 of 1,000 MATH outputs still contain no parseable answer (probably unfinished solutions), a count that shrinks with rank (543 → 473 → 456).
- **Caveat.** One seed and n = 1,000 give a binomial standard error of ≈1.2–1.4 pts (GSM8K) and ≈0.6–0.7 pts (MATH). The GSM8K trend is clear; adjacent-rank MATH gaps (0.8 pts ≈ 1 SE) are only suggestive, whereas 16→256 (1.6 pts) is ≈1.7 SE.

Question 2 (1%): Training time vs. target rank

(a) Following Question 1, record the training time (minutes) for target ranks 16, 64 and 256. (b) Provide observations and explanations on the training-time trends with different target ranks.

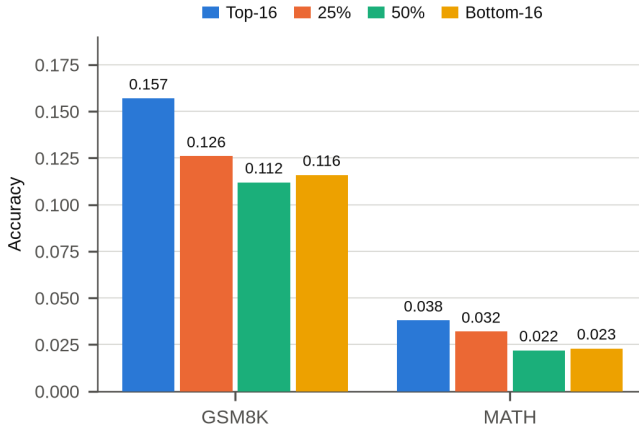
	Fine-Tuned (RANK = 16)	Fine-Tuned (RANK = 64)	Fine-Tuned (RANK = 256)
Training Time (min.)	66.12	71.43	96.90
Relative to RANK = 16	1.00×	1.08×	1.47×
Trainable parameters	11.3M	45.1M	180.4M

(b) Observations and explanations

- **Time grows with rank, but sub-linearly.** Quadrupling the rank (16→64) costs only +5.3 min (+8%); a 16× rank (16→256) costs +30.8 min (+47%), although the trainable parameters grow 16×.
- **Explanation.** Each PiSSA layer computes $y = xW_{res}^T + (xB^T)A^T$. The frozen W_{res} matmuls (forward, plus the backward pass that propagates activation gradients) cost the same at every rank and dominate at small r, while the adapter adds only $r(d_{in}+d_{out})$ multiply-adds per token per layer, i.e. 1.2% / 4.6% / 18.5% of the 0.97B dense projection weights at r = 16 / 64 / 256. The extra time at r = 256 exceeds this FLOP share; we attribute it (not profiled) to less efficient skinny matmuls and the optimizer / gradient handling of 16× more trainable parameters.
- **Notes.** The one-time SVD initialisation of the 112 matrices takes ≈2.4–2.5 min (143–149 s) at every rank and is reported separately from the training time. The four rank-16 runs of Q3 train for 65.8–66.3 min, so cost depends on the rank, not on which singular values are selected.

Question 3 (1%): Which singular values to fine-tune (rank = 16)

(a) For each dataset, perform SVD and fine-tune with four ranges of singular values at target rank 16 — Top-16 (component = default), 16 consecutive values starting at the 25% position, starting at the 50% position, and the Bottom-16 — and report the accuracy in bar charts. (b) Provide observations and explanations on the performance trends for both datasets.



The four singular-value ranges (rank = 16)

	Top-16	25%	50%	Bottom-16
GSM8K	0.157	0.126	0.112	0.116
MATH	0.038	0.032	0.022	0.023
Train loss†	0.455	0.495	0.497	0.507
Time (min.)	66.1	65.8	66.3	66.1

† Loss at the last logged step (single value, noisy).

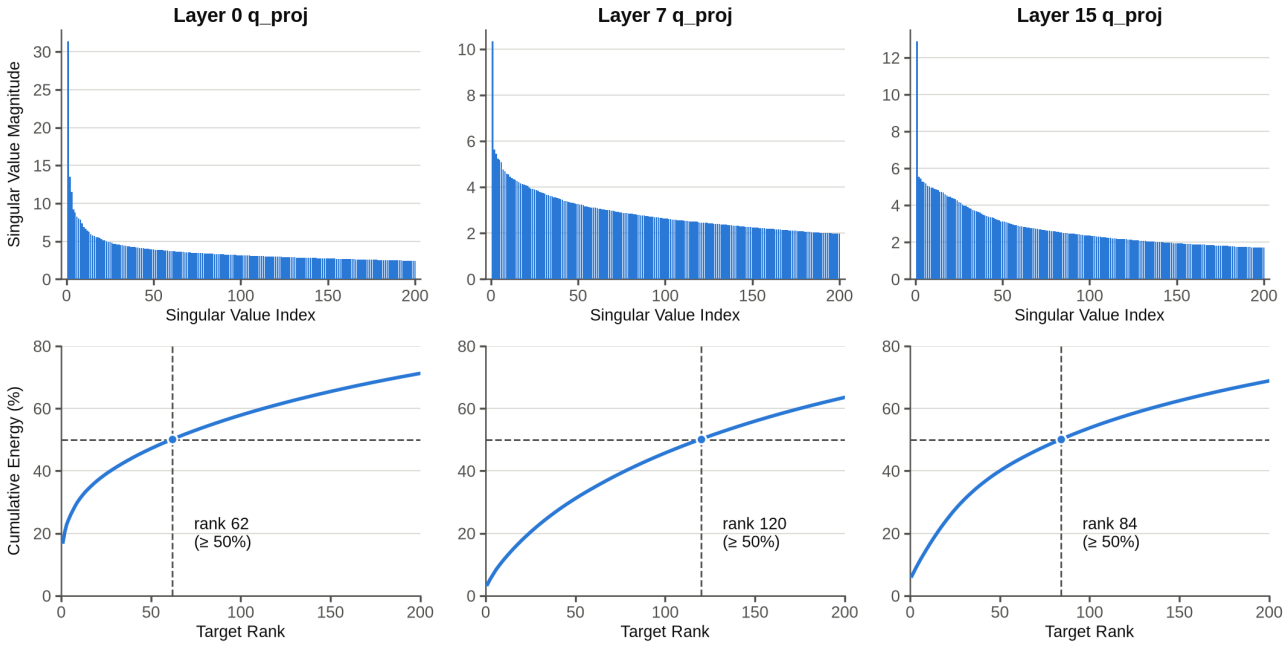
(b) Observations and explanations

- **Top-16 is best on both datasets:** GSM8K 0.157 vs. 0.112–0.126 (+3.1 to +4.5 pts, +25% to +40% relative) and MATH 0.038 vs. 0.022–0.032. This gap is ≈2–3 standard errors of the difference. The other three ranges lie within 1.4 pts (GSM8K) and 1.0 pt (MATH) of each other, comparable to the sampling noise (≈±1 pt), and are not ordered by position (25% > Bottom > 50%), so we do not read a trend among them.
- **Why the principal components win.** All four runs start from the same function (W_{res}+AB = W; max. reconstruction error ≈2×10^{-4}), so the difference comes only from which subspace is trained. With $A = U_r S_r^{1/2}$ and $B = S_r^{1/2} V_r^T$, a given parameter step changes W roughly in proportion to √σ. In q_proj (Q4 spectra) the top-16 values have σ ≈ 4–31 and hold 15–35% of the layer's energy, whereas the 25% / 50% bands have σ ≈ 0.95–1.4 / 0.43–0.62 (≈0.5–0.65% / 0.1–0.13% of the energy) and the bottom 16 have $\sigma < 4\times10^{-3}$ (≈10^{-6} %). Training the principal subspace therefore reshapes the dominant directions of each weight and lowers the loss faster (0.455 vs. 0.495–0.507), consistent with PiSSA's motivation; the other ranges behave more like a LoRA-style, near-zero-initialised adapter.
- **Every variant still beats the original model** (GSM8K 0.035, MATH 0.001), even Bottom-16, so part of the gain does not depend on the subspace: e.g. all four runs eliminate unparsed GSM8K outputs (15 → 0), i.e. they learn the answer format.

Question 4 (2%): SVD of q_proj in the original Llama-3.2-1B (no fine-tuning)

(a) Perform SVD on the q_proj weights of layers 0, 7, 15. (b) Report, for each layer, the minimum target rank that retains at least 50% cumulative energy (energy of rank $r = \Sigma_{i\le r}\sigma_i^2 / \Sigma_{all}\sigma_i^2$). (c) Visualise the first 200 singular values per layer: cumulative energy (line chart) and singular-value magnitude (bar chart). (d) Provide your observations.

	Layer 0	Layer 7	Layer 15
Minimum target rank for ≥ 50% energy	**62**	**120**	**84**
Cumulative energy at r = 16 / 64 / 200 (%)	34.8 / 50.6 / 71.2	15.4 / 35.9 / 63.5	21.0 / 44.7 / 68.8
σ_1 (share of total energy)	31.4 (17.2%)	10.3 (3.7%)	12.9 (6.4%)
Rank needed for 90% energy	519	584	548



Top row: singular-value magnitudes (bar charts); bottom row: cumulative energy (line charts, dashed lines mark 50% energy and the minimum rank). First 200 singular values of each layer's q_proj (2048 × 2048).

(d) Observations

- **q_proj is only weakly low-rank.** The spectrum has a steep head (layer 0: σ_1 = 31.4, σ_2 = 13.5, σ_{16} = 5.7) but then decays slowly: 50% of the energy needs 62 / 120 / 84 singular values (3.0% / 5.9% / 4.1% of 2,048), 90% needs 519–584 (25–29%), and the first 200 hold only 63–71%. The smallest singular values (≈6×10^{-6} to 8×10^{-5}) are non-zero, so the matrices are numerically full-rank.
- **Energy concentration differs by layer and is not monotonic in depth.** Layer 0 is the most concentrated (σ_1 alone holds 17.2%; min. rank 62), layer 7 the flattest (3.7%; min. rank 120) and layer 15 lies in between (6.4%; 84). All cumulative-energy curves are concave and still rising at rank 200.
- **Implications.** (i) One global rank keeps different fractions per layer (r = 64 keeps 51% / 36% / 45% in layers 0 / 7 / 15), so the middle layer needs a larger rank for the same fidelity. (ii) Ranks 16 / 64 / 256 cover only ≈15–35% / 36–51% / 70–76% of q_proj's energy, in line with Q1 where accuracy kept improving up to r = 256. (iii) The Top-16 values hold 15–35% of the energy while 16-wide bands at 25% / 50% / bottom hold ≈0.5–0.65% / 0.1–0.13% / ≈10^{-6} %, matching the ordering in Q3. These are indicative only, as Q4 looks at q_proj of three layers while Q1/Q3 adapt all 7 matrix types in 16 layers.